

開始使用 Cloud Spanner 開發遊戲

來源：<https://codelabs.developers.google.com/codelabs/cloud-spanner-getting-started-game?hl=zh-tw>

步驟數：9

在本程式碼研究室中，您將實作玩家個人資料服務和遊戲配對服務，以便與 Cloud Spanner 搭配使用。

目錄

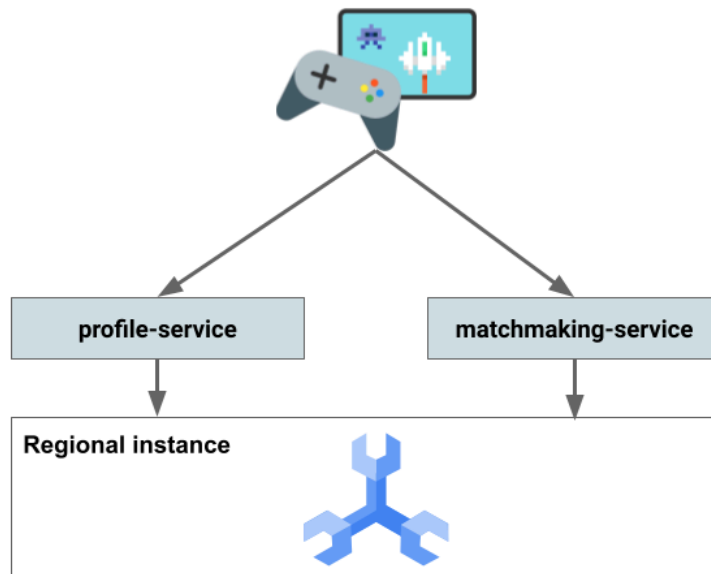
1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 建立 Spanner 執行個體和資料庫](#)
4. [4. 部署設定檔服務](#)
5. [5. 部署媒合服務](#)
6. [6. 開始答題](#)
7. [7. 擷取遊戲統計資料](#)
8. [8. 清除 \(選用\)](#)
9. [9. 恭喜！](#)

1. 簡介

[Cloud Spanner](#) 是全代管的關聯式資料庫服務，可水平擴充並在全球各地部署，提供 ACID 交易和 SQL 語意，同時維持效能和高可用性。

因此，如果遊戲架構要啟用全球玩家群，或擔心資料一致性問題，Spanner 就是絕佳選擇

在本實驗室中，您會建立兩項與區域 Spanner 資料庫互動的 Go 服務，讓玩家註冊並開始玩遊戲。



Google

Confidential + Proprietary

接著，您將運用 Python 負載架構 [Locust.io](#) 產生資料，模擬玩家註冊及玩遊戲。接著，您會查詢 Spanner，判斷有多少玩家正在玩遊戲，以及玩家勝場數與賽場數的相關統計資料。

最後，您將清除在本實驗室中建立的資源。

建構項目

本實驗室的學習內容包括：

- 可建立 Spanner 執行個體
- 部署以 Go 編寫的 Profile 服務，處理玩家註冊程序
- 部署以 Go 編寫的 Matchmaking 服務，將玩家指派給遊戲、決定贏家，並更新玩家的遊戲統計資料。

課程內容

- 如何設定 Cloud Spanner 執行個體
- 如何建立遊戲資料庫和結構定義
- 如何部署 Go 應用程式，以便與 Cloud Spanner 搭配使用
- 如何使用 [Locust](#) 產生資料
- 如何查詢 Cloud Spanner 中的資料，回答有關遊戲和玩家的問題。

軟硬體需求

- 已連結至帳單帳戶的 Google Cloud 專案。

- 網路瀏覽器，例如 [Chrome](#) 或 [Firefox](#)。
-

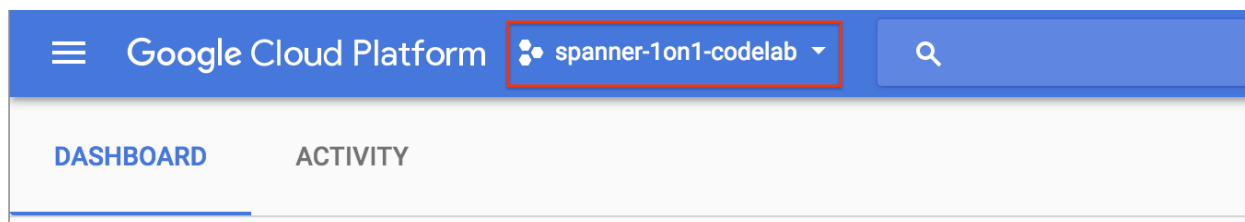
步驟 2 · 約 15 分鐘

2. 設定和需求

建立專案

如果您沒有 Google 帳戶 (Gmail 或 Google 應用程式)，請先[建立帳戶](#)。登入 Google Cloud Platform 主控台 (console.cloud.google.com)，然後建立新專案。

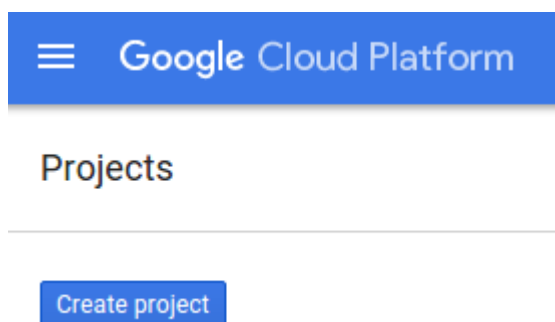
如果您已有專案，請按一下主控台左上方的專案選取下拉式選單：



然後在隨即顯示的對話方塊中，按一下「NEW PROJECT」(新專案) 按鈕，建立新專案：



如果您還沒有專案，應該會看到如下對話方塊，請建立第一個專案：



在隨後的專案建立對話方塊中，您可以輸入新專案的詳細資料：

New Project

Project name ?

spanner-1on1-codelab

Your project ID will be spanner-1on1-codelab ? [Edit](#)

Billing account ?

Choose account

Create

Cancel

請記住專案 ID，所有 Google Cloud 專案的專案 ID 都是不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 PROJECT_ID。

接下來，如果尚未啟用，請在開發人員控制台中[啟用帳單](#)，以便使用 Google Cloud 資源，並[啟用 Cloud Spanner API](#)。

Billing

This project has no billing account

This project is not linked to a billing account

[Link a billing account](#)

or

[Manage billing accounts](#)

完成本程式碼研究室的費用不應超過數美元，但如果您決定使用更多資源，或是將資源繼續執行 (請參閱本文件結尾的「清除」一節)，則可能會增加費用。Google Cloud Spanner 的定價說明文件請參閱[這裡](#)。

Google Cloud Platform 新使用者享有價值 **\$300 美元的免費試用期**，因此本程式碼研究室應完全免費。

設定 Google Cloud Shell

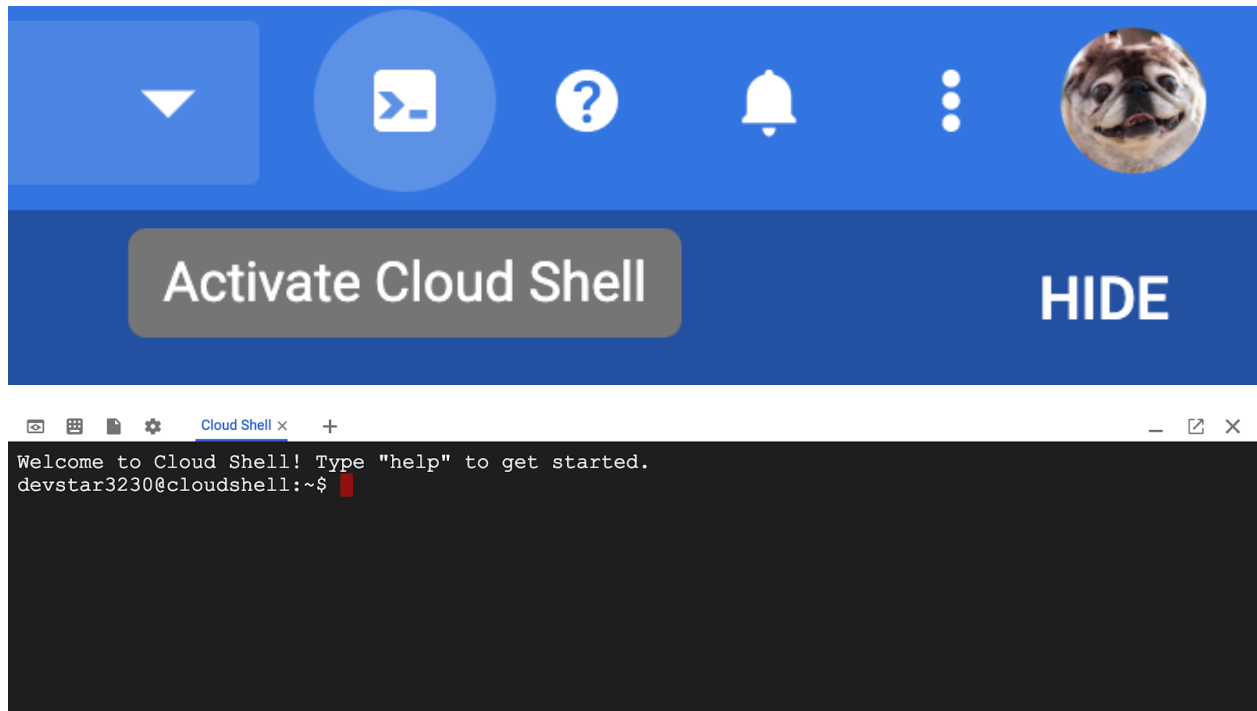
雖然可以透過筆電遠端操作 Google Cloud 和 Spanner，但在本程式碼研究室中，我們將使用 [Google Cloud Shell](#)，這是在雲端執行的指令列環境。

這部以 Debian 為基礎的虛擬機器，搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。也就是說，您只需要瀏覽器 (Chromebook 也可以) 就能完成本程式碼研究室。

1. 如要從 Cloud Shell 啟動 Cloud Shell，只要按一下「啟用 Cloud Shell」圖示



，系統就會佈建並連線至環境，這項作業只需要幾分鐘。



連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的 PROJECT_ID。

```
gcloud auth list
```

指令輸出

```
Credentialed accounts:
- <myaccount>@<mydomain>.com (active)
```

注意：Cloud Shell 已預先安裝 gcloud 指令列工具。詳情請參閱 [gcloud 指令列工具總覽](#)。

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如果專案未設定，請發出下列指令：

```
gcloud config set project <PROJECT_ID>
```

想尋找專案 ID 嗎？請檢查您在設定步驟中使用的 ID，或在 Cloud 控制台資訊主頁中尋找：

 Google Cloud Platform 

DASHBOARD

ACTIVITY

RECOMMENDATIONS

 **Project info** 

Project name

my-project

Project ID

coral-bucksaw-280805

Project number

58646390136

[ADD PEOPLE TO THIS PROJECT](#)

[→ Go to project settings](#)

Cloud Shell 也會預設設定部分環境變數，這些變數在您執行後續指令時可能很有用。

```
echo $GOOGLE_CLOUD_PROJECT
```

指令輸出

```
<PROJECT_ID>
```

注意：如果在自己的機器上執行 gcloud，各個工作階段都會保留這些設定。但在 Cloud Shell 中，每次啟動新工作階段或重新連線時，都必須重新設定。

下載程式碼

在 Cloud Shell 中，您可以下載本實驗室的程式碼。這是以 [v0.1.0](#) 版本為基礎，因此請檢查該標記：

```
git clone https://github.com/cloudspannerecosystem/spanner-gaming-sample.git
cd spanner-gaming-sample/

# Check out v0.1.0 release
git checkout tags/v0.1.0 -b v0.1.0-branch
```

指令輸出

```
Cloning into 'spanner-gaming-sample'...
*snip*
Switched to a new branch 'v0.1.0-branch'
```

設定 Locust 負載產生器

Locust 是 Python 負載測試架構，可用於測試 REST API 端點。在本程式碼研究室中，我們將重點介紹「generators」目錄中的 2 種不同負載測試：

- **authentication_server.py**：包含建立玩家的任務，以及取得隨機玩家來模擬單點查詢。
- **match_server.py**：包含建立及關閉遊戲的工作。建立遊戲時，系統會指派 100 位目前未玩遊戲的隨機玩家。關閉遊戲會更新 games_played 和 games_won 統計資料，並允許將這些玩家指派給未來的遊戲。

如要在 Cloud Shell 中執行 Locust，您需要 Python 3.7 以上版本。Cloud Shell 內建 Python 3.9，因此您只需驗證版本即可：

```
python -V
```

指令輸出

```
Python 3.9.12
```

現在，您可以安裝 Locust 的必要條件。

```
pip3 install -r requirements.txt
```

指令輸出

```
Collecting locust==2.11.1
*snip*
Successfully installed ConfigArgParse-1.5.3 Flask-BasicAuth-0.2.0 Flask-Cors-3.0.10 brotli-1.0.9 gevent-21.12.0 geventhttpclient-2.0.2 greenlet-1.1.3 locust-2.11.1 msgpack-1.0.4 psutil-5.9.2 pyzmq-22.3.0 roundrobin-0.0.4 zope.event-4.5.0 zope.interface-5.4.0
```

現在，請更新 PATH，以便找到新安裝的 *locust* 二進位檔：

```
PATH=~/.local/bin:$PATH"
which locust
```

指令輸出

```
/home/<user>/.local/bin/locust
```

摘要

在本步驟中，您已設定專案 (如果沒有的話)、啟用 Cloud Shell，並下載本實驗室的程式碼。

最後，您會在實驗室稍後階段設定 Locust，產生負載。

下一步

接著，您將設定 Cloud Spanner 執行個體和資料庫。

步驟 3 · 約 5 分鐘

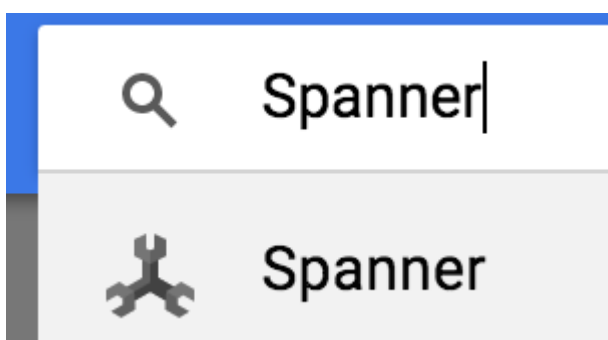
3. 建立 Spanner 執行個體和資料庫

建立 Spanner 執行個體

在本步驟中，我們會為程式碼研究室設定 Spanner 執行個體。在左上方的「漢堡」選單中搜尋 Spanner 項目



，或按下「/」並輸入「Spanner」來搜尋



接著，按一下

Create instance

並填寫表單，輸入執行個體的執行個體名稱 `cloudspanner-gaming`、選擇設定 (選取區域執行個體，例如 `us-central1`)，然後設定節點數量。在本程式碼研究室中，我們只需要 `500 processing units`。

最後，按一下「建立」，幾秒內您就能使用 Cloud Spanner 執行個體。

Instance name *

cloudspanner-gaming

Name must be 4–30 characters long

Instance ID *

cloudspanner-gaming

Lowercase letters, numbers and hyphens allowed

Choose a configuration

Determines where your nodes and data are located. Affects cost, performance and replication. A multi-region configuration will select the default leader region for your leader replicas. You can change your leader region at any time with a DDL statement. [Learn more](#)

[COMPARE REGION CONFIGURATIONS](#)

☒ Regional

☐ Multi-region

us-central1 (Iowa)

Allocate compute capacity

Your compute capacity determines the amount of data throughput, queries per second (QPS) and storage limits in your instance. One node equals 1,000 processing units. Affects billing.

Unit *

Processing units

Quantity *

500

Integers only. Enter in increments of 100 up to 1,000, followed by increments of 1,000.

✓ [COMPUTE CAPACITY GUIDANCE](#)

CREATE

CANCEL

建立資料庫和結構定義

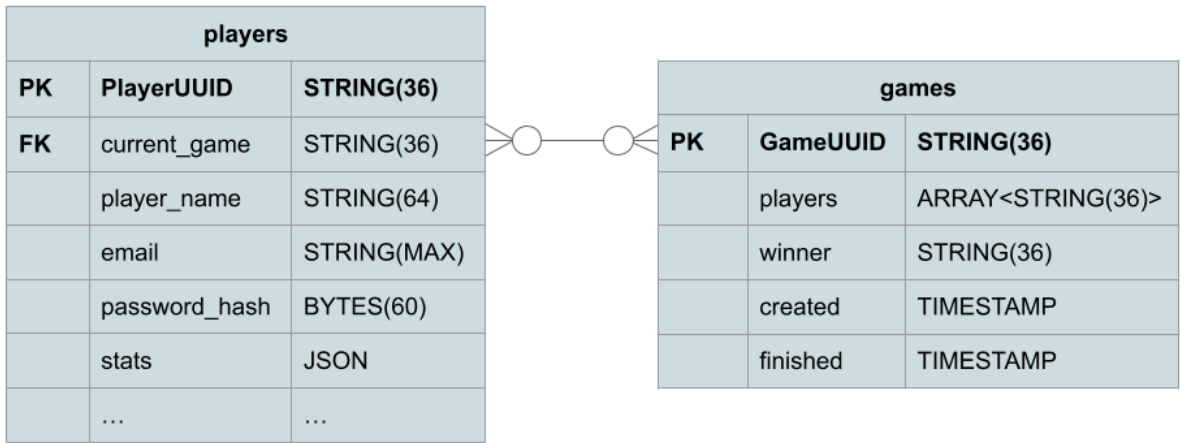
執行個體啟動後，您就可以建立資料庫。Spanner 允許單一執行個體有多個資料庫。

您可以在資料庫中定義結構定義。您也可以控管資料庫的存取權、設定自訂加密、設定最佳化工具，以及設定保留期限。

在多區域執行個體上，您也可以設定預設主要區域。如要進一步瞭解 Spanner 中的資料庫，請參閱[這篇文章](#)。

在本程式碼研究室中，您將使用預設選項建立資料庫，並在建立時提供結構定義。

本實驗室會建立兩個資料表：**players** 和 **games**。



玩家可以長期參與多場遊戲，但一次只能參與一場。玩家也有 *stats* 做為 [JSON 資料型態](#)，可追蹤有趣的統計資料，例如 *games_played* 和 *games_won*。由於日後可能會新增其他統計資料，因此這實際上是玩家的無結構定義資料欄。

遊戲會使用 Spanner 的 [ARRAY 資料類型](#)追蹤參與的玩家。遊戲結束後，系統才會填入遊戲的贏家和完成屬性。

有一個外鍵可確保玩家的 *current_game* 是有效遊戲。

現在請在執行個體總覽中點選「建立資料庫」，建立資料庫：

Overview

Name	cloudspanner-gaming
ID	cloudspanner-gaming
Configuration	us-central1 (Iowa)

Compute capacity ?

Processing units: 500

Nodes: 0

CPU utilisation (mean)

—

Databases

[+ CREATE DATABASE](#)

然後填寫詳細資料。重要選項包括資料庫名稱和方言。在本範例中，我們將資料庫命名為 **sample-game**，並選擇 Google 標準 SQL 方言。

警告：您必須使用 Google 標準 SQL 方言，本程式碼研究室才能運作。

在「結構定義」部分，複製並貼上[這個 DDL](#) 到方塊中：

```
CREATE TABLE games (  
  gameUUID STRING(36) NOT NULL,  
  players ARRAY<STRING(36)> NOT NULL,  
  winner STRING(36),  
  created TIMESTAMP,  
  finished TIMESTAMP,  
) PRIMARY KEY(gameUUID);  
  
CREATE TABLE players (  
  playerUUID STRING(36) NOT NULL,  
  player_name STRING(64) NOT NULL,  
  email STRING(MAX) NOT NULL,  
  password_hash BYTES(60) NOT NULL,  
  created TIMESTAMP,  
  updated TIMESTAMP,  
  stats JSON,  
  account_balance NUMERIC NOT NULL DEFAULT (0.00),  
  is_logged_in BOOL,  
  last_login TIMESTAMP,  
  valid_email BOOL,  
  current_game STRING(36),  
  FOREIGN KEY (current_game) REFERENCES games (gameUUID),  
) PRIMARY KEY(playerUUID);  
  
CREATE UNIQUE INDEX PlayerAuthentication ON players(email) STORING (password_hash);  
  
CREATE INDEX PlayerGame ON players(current_game);  
  
CREATE UNIQUE INDEX PlayerName ON players(player_name);
```

接著按一下「建立」按鈕，等待幾秒鐘，資料庫就會建立完成。

建立資料庫頁面應如下所示：

← Create a database in Cymbal Games spanner instance

Name your database

Enter a permanent name for your database of at least two characters, starting with a letter.

Database name *

sample-game

Lowercase letters, numbers, hyphens, underscores allowed

Select database dialect

Choose between Google Standard SQL and PostgreSQL dialects for your Spanner database.

☒ Google Standard SQL

☐ PostgreSQL

Define your schema (optional)

Add Spanner data definition language SQL statements below. Separate statements with a semicolon. [Learn more](#)

DDL TEMPLATES ▾

SHORTCUTS

Press Alt+F1 for accessibility options.

```
1 CREATE TABLE games (  
2   gameUUID STRING(36) NOT NULL,  
3   players ARRAY<STRING(36)> NOT NULL,  
4   winner STRING(36),  
5   created TIMESTAMP,  
6   finished TIMESTAMP,  
7 ) PRIMARY KEY(gameUUID);  
8  
9 CREATE TABLE players (  
10  playerUUID STRING(36) NOT NULL,  
11  player_name STRING(64) NOT NULL,  
12  email STRING(MAX) NOT NULL,  
13  password_hash BYTES(60) NOT NULL,  
14  created TIMESTAMP,
```

✓ SHOW ENCRYPTION OPTIONS

CREATE

CANCEL

現在，您需要在 Cloud Shell 中設定一些環境變數，供程式碼研究室稍後使用。因此請記下執行個體 ID，並在 Cloud Shell 中設定 INSTANCE_ID 和 DATABASE_ID

Overview

Name	cloudspanner-gaming
ID	cloudspanner-gaming
Configuration	us-central1 (Iowa)

```
export SPANNER_PROJECT_ID=$GOOGLE_CLOUD_PROJECT
export SPANNER_INSTANCE_ID=cloudspanner-gaming
export SPANNER_DATABASE_ID=sample-game
```

警告：請務必將 <INSTANCE_ID> 替換為實際的執行個體 ID。

摘要

您已在這個步驟中建立 Spanner 執行個體和 **sample-game** 資料庫。您也定義了這個範例遊戲使用的結構定義。

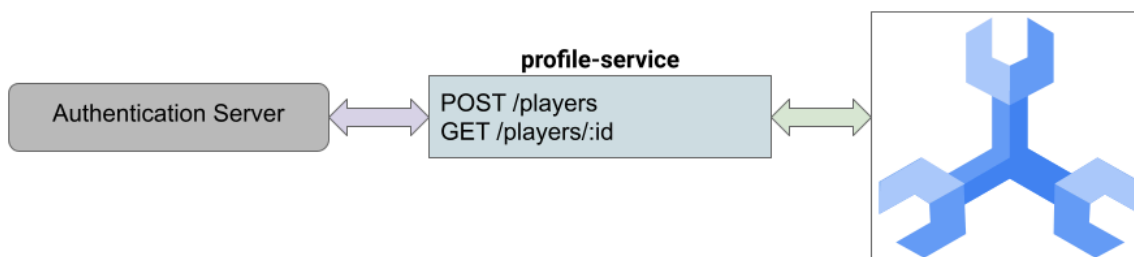
下一步

接著，您將部署設定檔服務，讓玩家註冊並開始玩遊戲！

4. 部署設定檔服務

服務總覽

設定檔服務是以 Go 撰寫的 REST API，採用 gin 架構。



Google

Confidential + Proprietary

玩家可透過這項 API 註冊玩遊戲。這項作業是透過簡單的 POST 指令建立，可接受玩家名稱、電子郵件地址和密碼。密碼會以 **bcrypt** 加密，雜湊值則會儲存在資料庫中。

電子郵件地址會視為專屬 ID，而玩家名稱則用於遊戲顯示。

這個 API 目前不會處理登入作業，但您可以自行實作這項功能，做為額外練習。

個人資料服務的 [./src/golang/profile-service/main.go](/src/golang/profile-service/main.go) 檔案會公開兩個主要端點，如下所示：

```
func main() {
    configuration, _ := config.NewConfig()

    router := gin.Default()
    router.SetTrustedProxies(nil)

    router.Use(setSpannerConnection(configuration))

    router.POST("/players", createPlayer)
    router.GET("/players", getPlayerUUIDs)
    router.GET("/players/:id", getPlayerByID)

    router.Run(configuration.Server.URL())
}
```

注意：本程式碼研究室不會使用 *getPlayerUUIDs* 函式和 *GET /players* 端點，因此下文不會討論這些內容。

這些端點的程式碼會路由至播放器模型。

```
func getPlayerByID(c *gin.Context) {
    var playerUUID = c.Param("id")

    ctx, client := getSpannerConnection(c)

    player, err := models.GetPlayerByUUID(ctx, client, playerUUID)
    if err != nil {
        cIndentedJSON(http.StatusNotFound, gin.H{"message": "player not found"})
        return
    }

    cIndentedJSON(http.StatusOK, player)
}

func createPlayer(c *gin.Context) {
    var player models.Player

    if err := c.BindJSON(&player); err != nil {
        c.AbortWithError(http.StatusBadRequest, err)
        return
    }

    ctx, client := getSpannerConnection(c)
    err := player.AddPlayer(ctx, client)
    if err != nil {
        c.AbortWithError(http.StatusBadRequest, err)
        return
    }

    cIndentedJSON(http.StatusCreated, player.PlayerUUID)
}
```

服務首先會設定 Spanner 連線。這項功能是在服務層級實作，可為服務建立工作階段集區。

```
func setSpannerConnection() gin.HandlerFunc {
    ctx := context.Background()
    client, err := spanner.NewClient(ctx, configuration.Spanner.URL())

    if err != nil {
        log.Fatal(err)
    }

    return func(c *gin.Context) {
        c.Set("spanner_client", *client)
        c.Set("spanner_context", ctx)
        c.Next()
    }
}
```

Player 和 *PlayerStats* 是定義如下的結構體：

```
type Player struct {
    PlayerUUID      string `json:"playerUUID" validate:"omitempty,uuid4"`
    Player_name     string `json:"player_name" validate:"required_with=Password Email"`
    Email           string `json:"email" validate:"required_with=Player_name Password,email"`
    // not stored in DB
    Password        string `json:"password" validate:"required_with=Player_name Email"`
    // stored in DB
    Password_hash   []byte `json:"password_hash"`
    created         time.Time
    updated         time.Time
    Stats           spanner.NullJSON `json:"stats"`
    Account_balance big.Rat          `json:"account_balance"`
    last_login      time.Time
    is_logged_in    bool
    valid_email     bool
    Current_game    string `json:"current_game" validate:"omitempty,uuid4"`
}

type PlayerStats struct {
    Games_played spanner.NullInt64 `json:"games_played"`
    Games_won    spanner.NullInt64 `json:"games_won"`
}
```

注意：Golang 有一個驗證器，當值繫結至 *Player* 結構體時，就會使用這個驗證器。這個結構體會運用必要欄位，以及 uuid4 和電子郵件驗證。您甚至可以使用 *omitempty*，只有在有值時驗證選填欄位。

新增玩家的函式會在 ReadWrite 交易中運用 DML 插入，因為新增玩家是單一陳述式，而非批次插入。函式如下所示：

```

func (p *Player) AddPlayer(ctx context.Context, client spanner.Client) error {
    // Validate based on struct validation rules
    err := p.Validate()
    if err != nil {
        return err
    }

    // take supplied password+salt, hash. Store in user_password
    passHash, err := hashPassword(p.Password)

    if err != nil {
        return errors.New("Unable to hash password")
    }

    p.Password_hash = passHash

    // Generate UUIDv4
    p.PlayerUUID = generateUUID()

    // Initialize player stats
    emptyStats := spanner.NullJSON{Value: PlayerStats{
        Games_played: spanner.NullInt64{Int64: 0, Valid: true},
        Games_won:    spanner.NullInt64{Int64: 0, Valid: true},
    }, Valid: true}

    // insert into spanner
    _, err = client.ReadWriteTransaction(ctx, func(ctx context.Context, txn
*spanner.ReadWriteTransaction) error {
        stmt := spanner.Statement{
            SQL: `INSERT players (playerUUID, player_name, email, password_hash, created,
stats) VALUES
                (@playerUUID, @playerName, @email, @passwordHash, CURRENT_TIMESTAMP(),
                @pStats)
            `,
            Params: map[string]interface{}{
                "playerUUID": p.PlayerUUID,
                "playerName": p.Player_name,
                "email":      p.Email,
                "passwordHash": p.Password_hash,
                "pStats":     emptyStats,
            },
        }

        _, err := txn.Update(ctx, stmt)
        return err
    })
    if err != nil {
        return err
    }
    // return empty error on success

```

```
    return nil
}
```

如要根據 UUID 擷取玩家，只要發出簡單的讀取要求即可。這會擷取玩家的 *playerUUID*、*player_name*、電子郵件地址和統計資料。

```
func GetPlayerByUUID(ctx context.Context, client spanner.Client, uuid string) (Player, error) {
    row, err := client.Single().ReadRow(ctx, "players",
        spanner.Key{uuid}, []string{"playerUUID", "player_name", "email", "stats"})
    if err != nil {
        return Player{}, err
    }

    player := Player{}
    err = row.ToStruct(&player)

    if err != nil {
        return Player{}, err
    }
    return player, nil
}
```

根據預設，服務會使用環境變數設定。請參閱 [./src/golang/profile-service/config/config.go](#) 檔案的相關章節。

```
func NewConfig() (Config, error) {
    *snip*
    // Server defaults
    viper.SetDefault("server.host", "localhost")
    viper.SetDefault("server.port", 8080)

    // Bind environment variable override
    viper.BindEnv("server.host", "SERVICE_HOST")
    viper.BindEnv("server.port", "SERVICE_PORT")
    viper.BindEnv("spanner.project_id", "SPANNER_PROJECT_ID")
    viper.BindEnv("spanner.instance_id", "SPANNER_INSTANCE_ID")
    viper.BindEnv("spanner.database_id", "SPANNER_DATABASE_ID")

    *snip*

    return c, nil
}
```

您可以看到，預設行為是在 *localhost:8080* 上執行服務。

有了這些資訊，就可以執行服務了。

執行設定檔服務

使用 `go` 指令執行服務。這會下載依附元件，並在通訊埠 8080 上建立執行中的服務：

```
cd ~/spanner-gaming-sample/src/golang/profile-service
go run . &
```

注意：下載 `golang` 依附元件可能需要幾分鐘。

警告：設定是透過本程式碼研究室稍早設定的 `SPANNER_` 環境變數處理。如果已重新連線至 Cloud Shell 終端機，請重設 `SPANNER_PROJECT_ID`、`SPANNER_INSTANCE_ID` 和 `SPANNER_DATABASE_ID`。

指令輸出：

```
[GIN-debug] [WARNING] Creating an Engine instance with the Logger and Recovery middleware
already attached.

[GIN-debug] [WARNING] Running in "debug" mode. Switch to "release" mode in production.
- using env:    export GIN_MODE=release
- using code:   gin.SetMode(gin.ReleaseMode)

[GIN-debug] POST    /players                --> main.createPlayer (4 handlers)
[GIN-debug] GET     /players                --> main.getPlayerUUIDs (4 handlers)
[GIN-debug] GET     /players/:id            --> main.getPlayerByID (4 handlers)
[GIN-debug] GET     /players/:id/stats      --> main.getPlayerStats (4 handlers)
[GIN-debug] Listening and serving HTTP on localhost:8080
```

發出 `curl` 指令來測試服務：

```
curl http://localhost:8080/players \
  --include \
  --header "Content-Type: application/json" \
  --request "POST" \
  --data '{"email": "test@gmail.com", "password": "s3cur3P@ss", "player_name": "Test
Player"}'
```

注意：這個資料庫不會儲存任何明文密碼。系統會使用 *bcrypt* 程式庫加密提供的密碼，然後將產生的輸出內容儲存在 **Players** 資料表中。

指令輸出：

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Date: <date> 18:55:08 GMT
Content-Length: 38
```

```
"506a1ab6-ee5b-4882-9bb1-ef9159a72989"
```

注意：每個要求的輸出 UUID 都會不同。

摘要

在這個步驟中，您部署了設定檔服務，讓玩家註冊並開始玩遊戲，並發出 POST API 呼叫來建立新玩家，藉此測試服務。

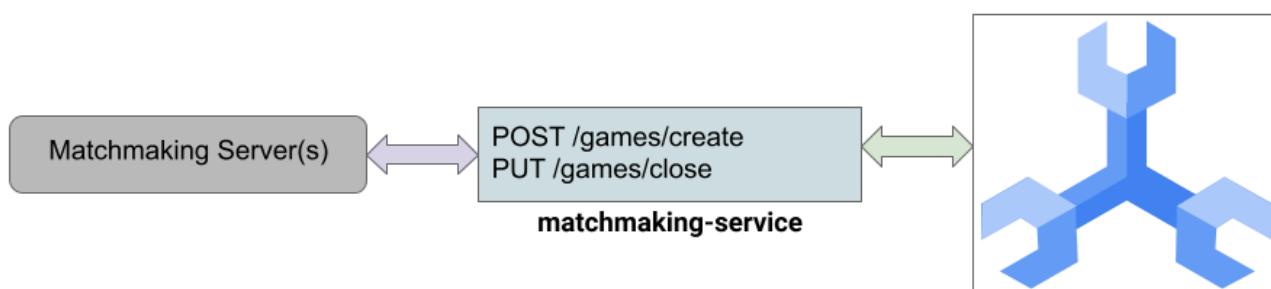
後續步驟

在下一個步驟中，您將部署媒合服務。

5. 部署媒合服務

服務總覽

媒合服務是以 Go 撰寫的 REST API，並採用 gin 架構。



Google

Confidential + Proprietary

在這個 API 中，遊戲會**建立**和**關閉**。建立遊戲時，系統會指派 10 位目前未玩遊戲的玩家。

遊戲**關閉**時，系統會隨機選出贏家，並調整每位玩家的 *games_played* 和 *games_won* 統計資料。此外，系統也會更新每位玩家的狀態，指出他們已停止遊戲，因此可參與未來的遊戲。

注意：一般來說，配對服務不會負責關閉遊戲。這裡納入這項功能，是為了說明如何在不讓架構變得複雜的情況下完成這項操作。

媒合服務的 [./src/golang/matchmaking-service/main.go](#) 檔案與 *profile* 服務的設定和程式碼類似，因此這裡不再重複說明。這項服務會公開兩個主要端點，如下所示：

```
func main() {
    router := gin.Default()
    router.SetTrustedProxies(nil)

    router.Use(setSpannerConnection())

    router.POST("/games/create", createGame)
    router.PUT("/games/close", closeGame)

    router.Run(configuration.Server.URL())
}
```

這項服務提供 **Game** 結構體，以及精簡的 **Player** 和 **PlayerStats** 結構體：

```
type Game struct {
    GameUUID string          `json:"gameUUID"`
    Players  []string                 `json:"players"`
    Winner   string                  `json:"winner"`
    Created  time.Time                `json:"created"`
    Finished spanner.NullTime       `json:"finished"`
}

type Player struct {
    PlayerUUID string          `json:"playerUUID"`
    Stats      spanner.NullJSON `json:"stats"`
    Current_game string        `json:"current_game"`
}

type PlayerStats struct {
    Games_played int `json:"games_played"`
    Games_won    int `json:"games_won"`
}
```

如要建立[遊戲](#)，配對服務會隨機選取 100 位目前未在玩遊戲的玩家。

我們選擇使用 [Spanner 突變](#) 建立遊戲並指派玩家，因為對於大型變更而言，突變的效能優於 DML。

```

// Create a new game and assign players
// Players that are not currently playing a game are eligible to be selected for the new game
// Current implementation allows for less than numPlayers to be placed in a game
func (g *Game) CreateGame(ctx context.Context, client spanner.Client) error {
    // Initialize game values
    g.GameUUID = generateUUID()

    numPlayers := 10

    // Create and assign
    _, err := client.ReadWriteTransaction(ctx, func(ctx context.Context, txn
*spanner.ReadWriteTransaction) error {
        var m []*spanner.Mutation

        // get players
        query := fmt.Sprintf("SELECT playerUUID FROM (SELECT playerUUID FROM players WHERE
current_game IS NULL LIMIT 10000) TABLESAMPLE RESERVOIR (%d ROWS)", numPlayers)
        stmt := spanner.Statement{SQL: query}
        iter := txn.Query(ctx, stmt)

        playerRows, err := readRows(iter)
        if err != nil {
            return err
        }

        var playerUUIDs []string

        for _, row := range playerRows {
            var pUUID string
            if err := row.Columns(&pUUID); err != nil {
                return err
            }

            playerUUIDs = append(playerUUIDs, pUUID)
        }

        // Create the game
        gCols := []string{"gameUUID", "players", "created"}
        m = append(m, spanner.Insert("games", gCols, []interface{}{g.GameUUID, playerUUIDs,
time.Now()}))

        // Update players to lock into this game
        for _, p := range playerUUIDs {
            pCols := []string{"playerUUID", "current_game"}
            m = append(m, spanner.Update("players", pCols, []interface{}{p, g.GameUUID}))
        }

        txn.BufferWrite(m)

        return nil
    })
}

```

```
    if err != nil {  
        return err  
    }  
  
    return nil  
}
```

系統會使用 GoogleSQL 的 [TABLESPACE RESERVOIR](#) 功能，以 SQL 隨機選取玩家。

注意：這種隨機指派玩家的方法效能不佳，因為需要掃描 *PlayerGame* 索引。理想情況下，隨機對戰服務在選擇要指派給遊戲的玩家時，會更加精細。不過，在處理 *Spanner* 時，這項功能可用於示範。

關閉遊戲的程序稍微複雜一點。包括從遊戲玩家中隨機選出贏家、標記遊戲結束時間，以及更新每位玩家的 *games_played* 和 *games_won* 統計資料。

由於這項複雜度和變更量，系統再次選擇突變來關閉遊戲。

```

func determineWinner(playerUUIDs []string) string {
    if len(playerUUIDs) == 0 {
        return ""
    }

    var winnerUUID string

    rand.Seed(time.Now().UnixNano())
    offset := rand.Intn(len(playerUUIDs))
    winnerUUID = playerUUIDs[offset]
    return winnerUUID
}

// Given a list of players and a winner's UUID, update players of a game
// Updating players involves closing out the game (current_game = NULL) and
// updating their game stats. Specifically, we are incrementing games_played.
// If the player is the determined winner, then their games_won stat is incremented.
func (g Game) updateGamePlayers(ctx context.Context, players []Player, txn
*spanner.ReadWriteTransaction) error {
    for _, p := range players {
        // Modify stats
        var pStats PlayerStats
        json.Unmarshal([]byte(p.Stats.String()), &pStats)

        pStats.Games_played = pStats.Games_played + 1

        if p.PlayerUUID == g.Winner {
            pStats.Games_won = pStats.Games_won + 1
        }
        updatedStats, _ := json.Marshal(pStats)
        p.Stats.UnmarshalJSON(updatedStats)

        // Update player
        // If player's current game isn't the same as this game, that's an error
        if p.Current_game != g.GameUUID {
            errorMsg := fmt.Sprintf("Player '%s' doesn't belong to game '%s'.", p.PlayerUUID,
g.GameUUID)
            return errors.New(errorMsg)
        }

        cols := []string{"playerUUID", "current_game", "stats"}
        newGame := spanner.NullString{
            StringVal: "",
            Valid:      false,
        }

        txn.BufferWrite([]*spanner.Mutation{
            spanner.Update("players", cols, []interface{}{p.PlayerUUID, newGame, p.Stats}),
        })
    }

    return nil
}

```

```

}

// Closing game. When provided a Game, choose a random winner and close out the game.
// A game is closed by setting the winner and finished time.
// Additionally all players' game stats are updated, and the current_game is set to null to
allow
// them to be chosen for a new game.
func (g *Game) CloseGame(ctx context.Context, client spanner.Client) error {
    // Close game
    _, err := client.ReadWriteTransaction(ctx,
        func(ctx context.Context, txn *spanner.ReadWriteTransaction) error {
            // Get game players
            playerUUIDs, players, err := g.getGamePlayers(ctx, txn)

            if err != nil {
                return err
            }

            // Might be an issue if there are no players!
            if len(playerUUIDs) == 0 {
                errorMsg := fmt.Sprintf("No players found for game '%s'", g.GameUUID)
                return errors.New(errorMsg)
            }

            // Get random winner
            g.Winner = determineWinner(playerUUIDs)

            // Validate game finished time is null
            row, err := txn.ReadRow(ctx, "games", spanner.Key{g.GameUUID},
                []string{"finished"})
            if err != nil {
                return err
            }

            if err := row.Column(0, &g.Finished); err != nil {
                return err
            }

            // If time is not null, then the game is already marked as finished.
            // That's an error.
            if !g.Finished.IsNull() {
                errorMsg := fmt.Sprintf("Game '%s' is already finished.", g.GameUUID)
                return errors.New(errorMsg)
            }

            cols := []string{"gameUUID", "finished", "winner"}
            txn.BufferWrite([]*spanner.Mutation{
                spanner.Update("games", cols, []interface{}{g.GameUUID, time.Now(),
                    g.Winner}),
            })

            // Update each player to increment stats.games_played

```

```

        // (and stats.games_won if winner), and set current_game
        // to null so they can be chosen for a new game
        playerErr := g.updateGamePlayers(ctx, players, txn)
        if playerErr != nil {
            return playerErr
        }

        return nil
    })

    if err != nil {
        return err
    }

    return nil
}

```

設定同樣是透過環境變數處理，詳情請參閱服務的 [./src/golang/matchmaking-service/config/config.go](#)。

```

// Server defaults
viper.SetDefault("server.host", "localhost")
viper.SetDefault("server.port", 8081)

// Bind environment variable override
viper.BindEnv("server.host", "SERVICE_HOST")
viper.BindEnv("server.port", "SERVICE_PORT")
viper.BindEnv("spanner.project_id", "SPANNER_PROJECT_ID")
viper.BindEnv("spanner.instance_id", "SPANNER_INSTANCE_ID")
viper.BindEnv("spanner.database_id", "SPANNER_DATABASE_ID")

```

為避免與設定檔服務發生衝突，這項服務預設會在 *localhost:8081* 上執行。

有了這些資訊，現在可以執行媒合服務了。

執行配對服務

使用 `go` 指令執行服務。這會建立在通訊埠 8082 上執行的服務。這項服務與設定檔服務有許多相同的依附元件，因此不會下載新的依附元件。

```

cd ~/spanner-gaming-sample/src/golang/matchmaking-service
go run . &

```

注意：下載 `golang` 依附元件可能需要幾分鐘。

警告：設定是透過本程式碼研究室稍早設定的 `SPANNER_` 環境變數處理。如果必須重新連線至 Cloud Shell 終端機，請重設 `SPANNER_PROJECT_ID`、`SPANNER_INSTANCE_ID` 和 `SPANNER_DATABASE_ID`。

指令輸出：

```
[GIN-debug] [WARNING] Creating an Engine instance with the Logger and Recovery middleware already attached.

[GIN-debug] [WARNING] Running in "debug" mode. Switch to "release" mode in production.
- using env:   export GIN_MODE=release
- using code:  gin.SetMode(gin.ReleaseMode)

[GIN-debug] POST    /games/create          --> main.createGame (4 handlers)
[GIN-debug] PUT     /games/close           --> main.closeGame (4 handlers)
[GIN-debug] Listening and serving HTTP on localhost:8081
```

建立遊戲

測試服務，建立遊戲。首先，在 Cloud Shell 中開啟新的終端機：



然後發出下列 curl 指令：

```
curl http://localhost:8081/games/create \
  --include \
  --header "Content-Type: application/json" \
  --request "POST"
```

指令輸出：

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Date: <date> 19:38:45 GMT
Content-Length: 38

"f45b0f7f-405b-4e67-a3b8-a624e990285d"
```

注意：複製 curl 指令輸出的 UUID。這項資訊會在下一個步驟中用於關閉遊戲。

結束遊戲

```
curl http://localhost:8081/games/close \  
  --include \  
  --header "Content-Type: application/json" \  
  --data '{"gameUUID": "f45b0f7f-405b-4e67-a3b8-a624e990285d"}' \  
  --request "PUT"
```

注意：請將 gameUUID 替換為 curl 指令的輸出內容。

指令輸出：

```
HTTP/1.1 200 OK  
Content-Type: application/json; charset=utf-8  
Date: <date> 19:43:58 GMT  
Content-Length: 38  
  
"506a1ab6-ee5b-4882-9bb1-ef9159a72989"
```

注意：這項指令輸出的 UUID 就是該場遊戲的贏家！

摘要

在這個步驟中，您已部署 matchmaking-service，負責建立遊戲並將玩家指派給該遊戲。這項服務也會處理遊戲結尾，隨機選出贏家，並更新所有遊戲玩家的 *games_played* 和 *games_won* 統計資料。

後續步驟

服務已開始運作，現在可以讓玩家註冊並暢玩遊戲！

6. 開始答題

設定檔和媒合服務現在已開始執行，您可以使用提供的 Locust 產生器產生負載。

Locust 提供網頁介面來執行產生器，但本實驗室會使用指令列 (*-headless* 選項)。

註冊玩家

首先，請產生玩家。

在 [./generators/authentication_server.py](#) 檔案中建立玩家的 Python 程式碼如下所示：

```
class PlayerLoad(HttpUser):
    def on_start(self):
        global pUUIDs
        pUUIDs = []

    def generatePlayerName(self):
        return ''.join(random.choices(string.ascii_lowercase + string.digits, k=32))

    def generatePassword(self):
        return ''.join(random.choices(string.ascii_lowercase + string.digits, k=32))

    def generateEmail(self):
        return ''.join(random.choices(string.ascii_lowercase + string.digits, k=32) + ['@'] +
                        random.choices(['gmail', 'yahoo', 'microsoft']) + ['.com'])

    @task
    def createPlayer(self):
        headers = {"Content-Type": "application/json"}
        data = {"player_name": self.generatePlayerName(), "email": self.generateEmail(),
               "password": self.generatePassword()}

        with self.client.post("/players", data=json.dumps(data), headers=headers,
                              catch_response=True) as response:
            try:
                pUUIDs.append(response.json())
            except json.JSONDecodeError:
                response.failure("Response could not be decoded as JSON")
            except KeyError:
                response.failure("Response did not contain expected key 'gameUUID'")
```

系統會隨機產生玩家名稱、電子郵件地址和密碼。

第二項工作會擷取成功註冊的玩家，以產生讀取負載。

```

@task(5)
def getPlayer(self):
    # No player UUIDs are in memory, reschedule task to run again later.
    if len(pUUIDs) == 0:
        raise RescheduleTask()

    # Get first player in our list, removing it to avoid contention from concurrent
requests
    pUUID = pUUIDs[0]
    del pUUIDs[0]

    headers = {"Content-Type": "application/json"}

    self.client.get(f"/players/{pUUID}", headers=headers, name="/players/[playerUUID]")

```

下列指令會呼叫 [./generators/authentication_server.py](#) 檔案，以一次兩個執行緒的並行數 (u=2)，為 30 秒 (t=30s) 生成新播放器：

```

cd ~/spanner-gaming-sample
locust -H http://127.0.0.1:8080 -f ./generators/authentication_server.py --headless -u=2 -r=2
-t=30s

```

注意：執行產生器時，locust 指令會輸出所發出要求的統計資料，包括要求數、失敗數和延遲時間資訊。

玩家加入遊戲

玩家註冊完畢後，就會想開始玩遊戲！

在 [./generators/match_server.py](#) 檔案中建立及關閉遊戲的 Python 程式碼如下：

```

from locust import HttpUser, task
from locust.exception import RescheduleTask

import json

class GameMatch(HttpUser):
    def on_start(self):
        global openGames
        openGames = []

    @task(2)
    def createGame(self):
        headers = {"Content-Type": "application/json"}

        # Create the game, then store the response in memory of list of open games.
        with self.client.post("/games/create", headers=headers, catch_response=True) as response:
            try:
                openGames.append({"gameUUID": response.json()})
            except json.JSONDecodeError:
                response.failure("Response could not be decoded as JSON")
            except KeyError:
                response.failure("Response did not contain expected key 'gameUUID'")

    @task
    def closeGame(self):
        # No open games are in memory, reschedule task to run again later.
        if len(openGames) == 0:
            raise RescheduleTask()

        headers = {"Content-Type": "application/json"}

        # Close the first open game in our list, removing it to avoid
        # contention from concurrent requests
        game = openGames[0]
        del openGames[0]

        data = {"gameUUID": game["gameUUID"]}
        self.client.put("/games/close", data=json.dumps(data), headers=headers)

```

執行這項產生器時，遊戲開啟和關閉的比例為 2:1。這項指令會執行產生器 10 秒 (**-t=10s**)：

```
locust -H http://127.0.0.1:8081 -f ./generators/match_server.py --headless -u=1 -r=1 -t=10s
```

摘要

在這個步驟中，您模擬玩家註冊玩遊戲，然後使用媒合服務模擬玩家玩遊戲。這些模擬作業運用 Locust Python 架構，向服務的 REST API 發出要求。

您可以隨意修改建立玩家和玩遊戲所花的時間，以及並行使用者人數 (**-u**)。

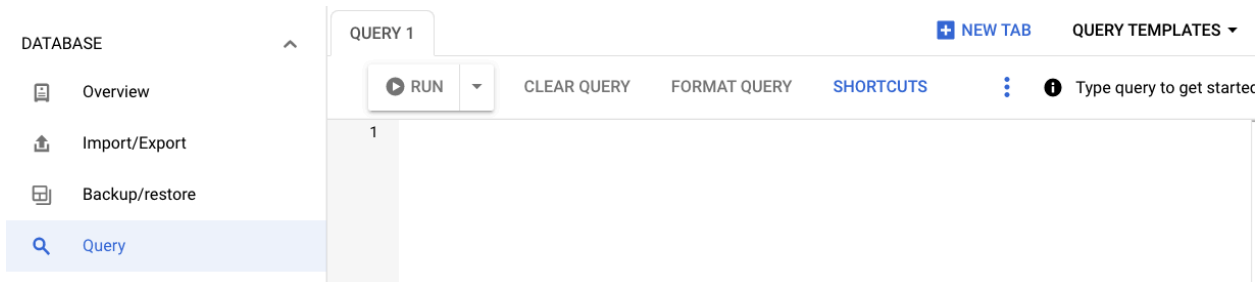
後續步驟

模擬完成後，您會想查詢 [Spanner](#)，查看各種統計資料。

7. 擷取遊戲統計資料

現在我們已模擬玩家註冊及玩遊戲，您應該查看統計資料。

如要執行這項操作，請使用 Cloud Console 向 Spanner 發出查詢要求。



檢查開放與封閉遊戲

如果遊戲已結束，系統會填入 *finished* 時間戳記；如果遊戲仍在進行中，*finished* 則會是 NULL。這個值會在遊戲關閉時設定。

因此，這項查詢可讓您檢查開啟和關閉的遊戲數量：

```
SELECT Type, NumGames FROM
(SELECT "Open Games" as Type, count(*) as NumGames FROM games WHERE finished IS NULL
UNION ALL
SELECT "Closed Games" as Type, count(*) as NumGames FROM games WHERE finished IS NOT NULL
)
```

結果：

Type	NumGames
Open Games	0
Closed Games	175

注意：開放式與封閉式遊戲的確切結果會有所不同。

檢查玩遊戲的玩家人數與未玩遊戲的玩家人數

如果玩家的 *current_game* 欄已設定，表示玩家正在玩遊戲。否則表示他們目前沒有在玩遊戲。

因此，如要比較目前正在玩遊戲和未玩遊戲的玩家人數，請使用下列查詢：

```
SELECT Type, NumPlayers FROM
(SELECT "Playing" as Type, count(*) as NumPlayers FROM players WHERE current_game IS NOT NULL
UNION ALL
SELECT "Not Playing" as Type, count(*) as NumPlayers FROM players WHERE current_game IS NULL
)
```

結果：

Type	NumPlayers
Playing	0
Not Playing	310

注意：玩家是否參與遊戲會影響確切結果。

決定得獎者

遊戲結束時，系統會隨機選取一位玩家做為贏家。系統會在遊戲結束時增加該玩家的 *games_won* 統計資料。

```
SELECT playerUUID, stats
FROM players
WHERE CAST(JSON_VALUE(stats, "$.games_won") AS INT64)>0
LIMIT 10;
```

結果：

playerUUID	stats
07e247c5-f88e-4bca-a7bc-12d2485f2f2b	{"games_played":49,"games_won":1}
09b72595-40af-4406-a000-2fb56c58fe92	{"games_played":56,"games_won":1}
1002385b-02a0-462b-a8e7-05c9b27223aa	{"games_played":66,"games_won":1}
13ec3770-7ae3-495f-9b53-6322d8e8d6c3	{"games_played":44,"games_won":1}
15513852-3f2a-494f-b437-fe7125d15f1b	{"games_played":49,"games_won":1}
17faec64-4f77-475c-8df8-6ab026cf6698	{"games_played":50,"games_won":1}

1abfcb27-037d-446d-bb7a-b5cd17b5733d	{"games_played":63,"games_won":1}
2109a33e-88bd-4e74-a35c-a7914d9e3bde	{"games_played":56,"games_won":2}
222e37d9-06b0-4674-865d-a0e5fb80121e	{"games_played":60,"games_won":1}
22ced15c-0da6-4fd9-8cb2-1ffd233b3c56	{"games_played":50,"games_won":1}

注意：實際的統計結果會有所不同。

警告：ORDER BY 無法用於 JSON 路徑，因此結果無法依「\$.games_won」排序。如要進一步瞭解這項限制和其他限制，請參閱「[JSON 限制](#)」。

摘要

在這個步驟中，您使用 Cloud 控制台查詢 Spanner，查看了各種玩家和遊戲的統計資料。

後續步驟

接著，請進行清理！

8. 清除 (選用)

警告：如要繼續進行下一個 [Cloud Spanner Game Trading Post](#) 程式碼研究室，請略過這個清除步驟。

如要清除資源，請前往 Cloud Console 的 [Cloud Spanner 專區](#)，然後刪除在 Codelab 步驟「設定 Cloud Spanner 執行個體」中建立的「cloudspanner-gaming」執行個體。

步驟 9 · 約 2 分鐘

9. 恭喜！

恭喜！您已成功在 Spanner 上部署範例遊戲

後續步驟

在本實驗室中，您已瞭解如何使用 Golang 驅動程式處理 Spanner 的各種主題。這有助於您進一步瞭解重要概念，例如：

- 結構定義設計
- DML 與變異
- 使用 Go 語言

如需使用 Spanner 做為遊戲後端的其他範例，請務必參閱 [Cloud Spanner 遊戲交易平台](#) 程式碼研究室！
